Verification Tutorial

- Verification Tutorial
- Introduction
- System Architecture
- Verification Flow
  - 1. TestCase Verify
    - prepare instruction hex file
  - 2. Interactive CLI
  - 3. Random Test (Only support RISC-V)
    - Random Test Examples
- Difftest Debug implementation

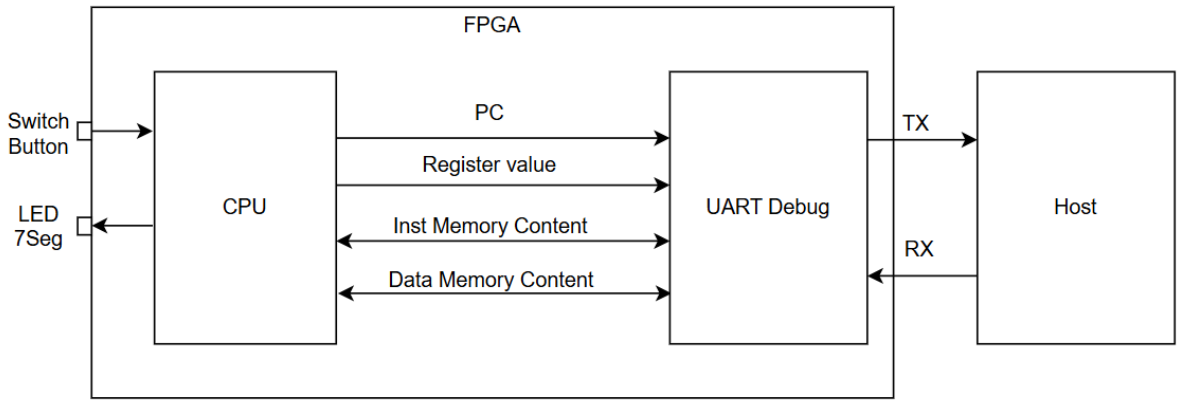
Introduction

This document introduces the CPU's automated verification flow and the code modification methods required to support automated verification.

Video Introduction: https://meeting.tencent.com/crm/23YBZYMq68
International students can click "cc" to select English subtitles.

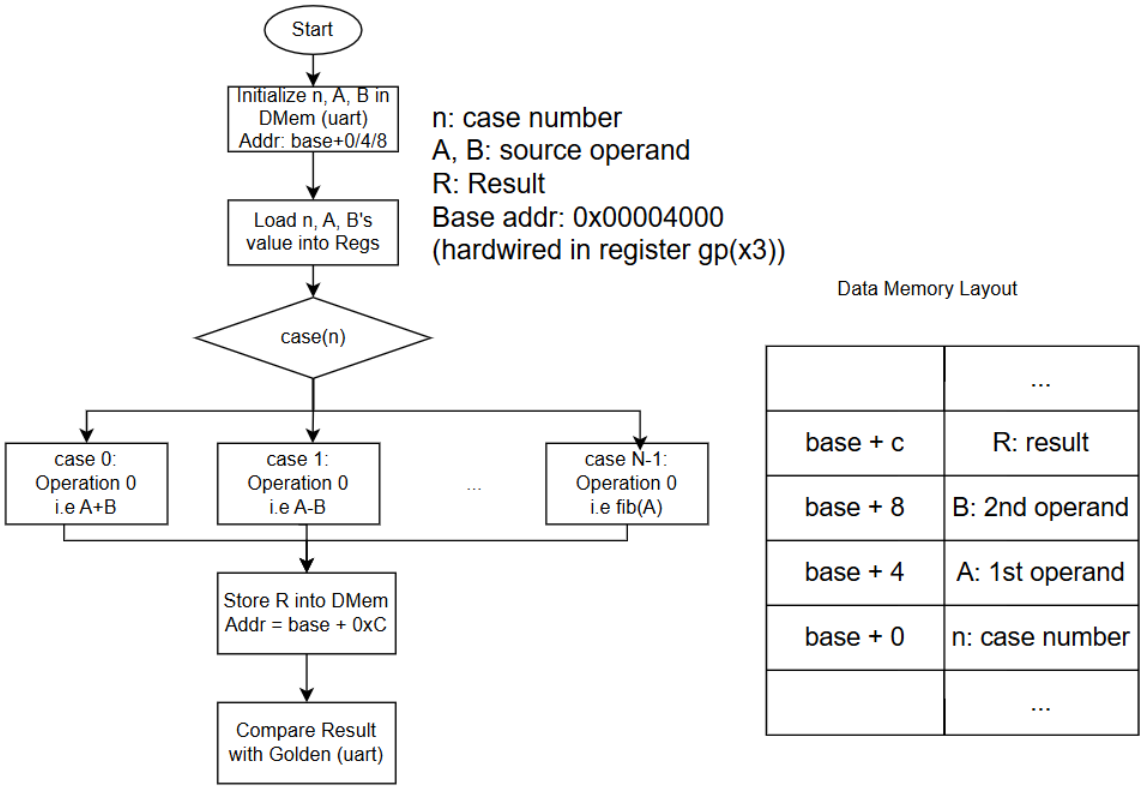
> by baiyh@sustech.edu.cn
>
> This Document and video are for internal use only at Southern University of Science and Technology. Please do not repost it on other public platforms without permission from the instructor.

System Architecture



Verification Flow

1. TestCase Verify



In this test, the host acts as both the stimulus generator and the arbiter, writing test case IDs and two operands into fixed memory addresses in the FPGA memory (Base+0, Base+4, Base+8) via UART. The RISC-V code running on the FPGA CPU reads the case ID, branches to execute a pre-written target assembly snippet, and performs the computation using the provided operands. Subsequently, it writes the result back to the output address (Base+C). Finally, the host reads back the actual execution result from this memory address via UART and compares it against the expected result, thereby achieving automated verification of the logical correctness of the underlying hardware instructions.

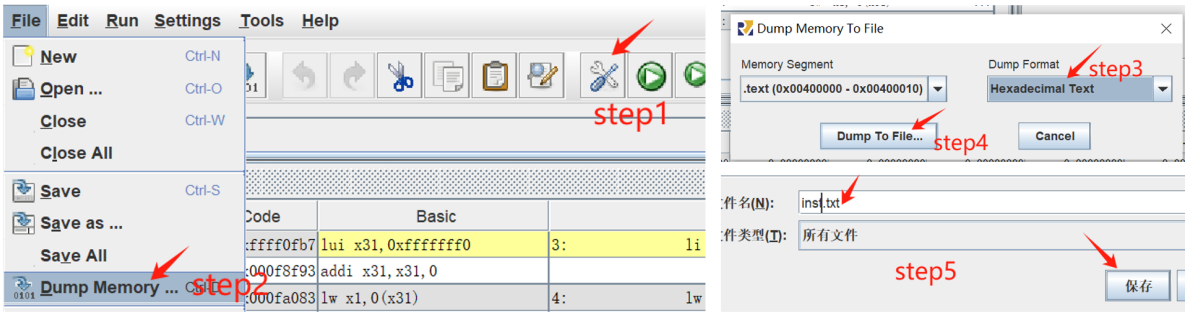
For example, suppose a student configures two test cases within the RISC-V assembly code (Case 0 and Case 1, as depicted in the previous figure), corresponding to the `add` and `sub` instructions, respectively. The hardware evaluation can be executed by supplying the following dataset:

```
0, [0xB, 0x1], 0xC
0, [11, 24], 35
1, [50, 20], 30
1, [0xFFFFFFFF, 0xFFFFFFFE], 0x1
```

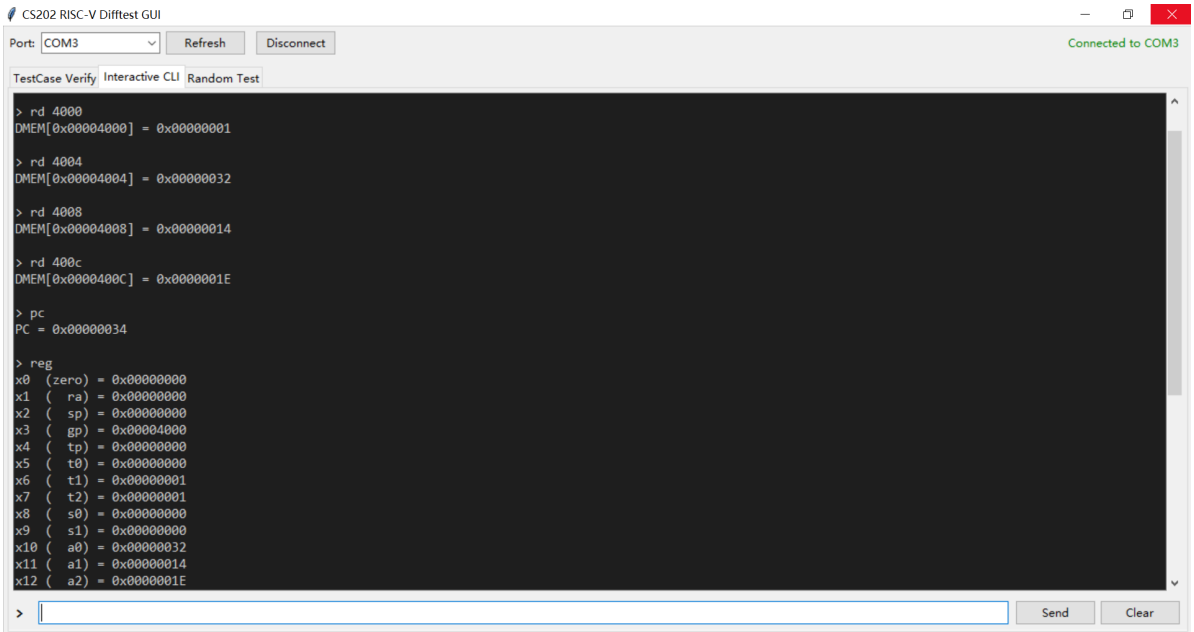
Taking the first entry as an example, the framework dictates the execution of Case 0 (addition) using operands `0xB` and `0x1`, and subsequently cross-verifies whether the hardware's computed output matches the expected value of `0xC`.

To start the test, you must first program the FPGA with the bitstream file. Next, transmit the assembled RISC-V machine code in hex format via UART to load the instructions into the CPU's memory.

prepare instruction hex file

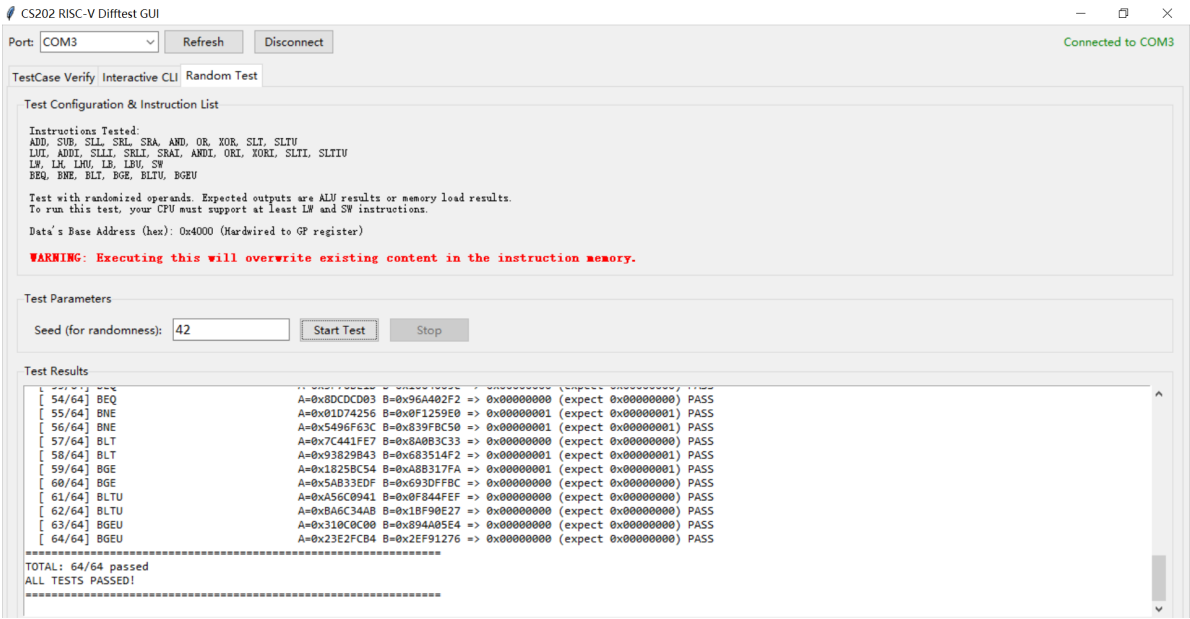


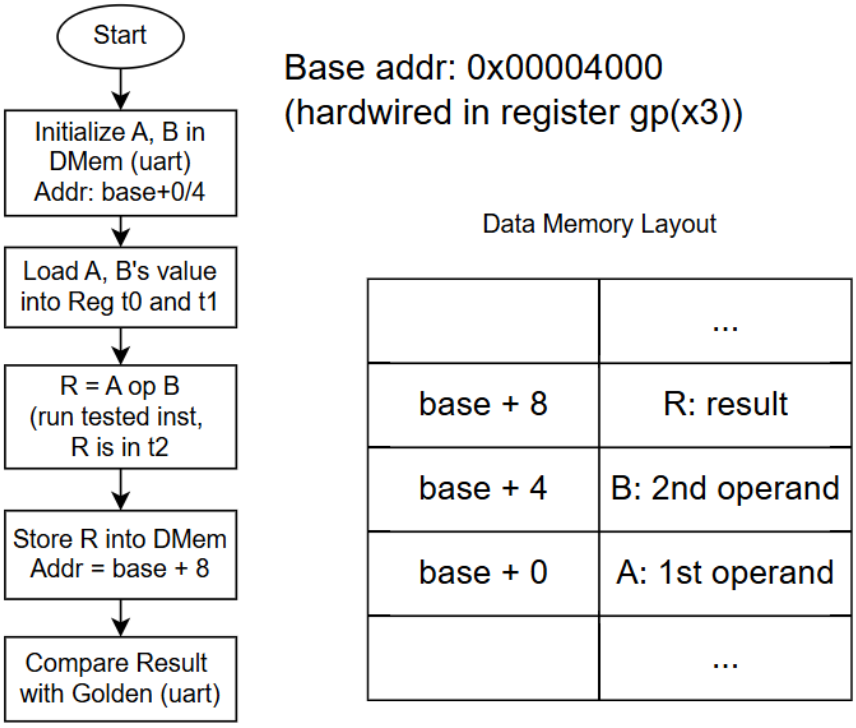
2. Interactive CLI



The CLI mode provides an interactive debugging interface for direct communication with the FPGA CPU via UART. You can execute commands such as `ping` to verify connectivity, `reset` / `run` / `halt` to control CPU execution, `step` to execute single instructions, and `reg` / `pc` to check registers and program counter's value. Additionally, commands like `wi` / `ri` / `wd` / `rd` allow you to directly read and write instruction memory and data memory at specified addresses.

3. Random Test (Only support RISC-V)





The random test mode automatically generates diverse test cases covering all major RV32I instructions. For each test, the framework generates random operands, assembles a corresponding test program with preamble and epilogue code, loads it into the CPU, executes it, and automatically verifies the result against the expected value computed by the host. Because the framework directly generates native machine code, this testing mode is exclusively applicable to the RISC-V architecture. Furthermore, to execute these automated tests, the CPU datapath must implement at least the `lw` and `sw` instructions.

This test is intended solely to help you quickly verify that your CPU datapath correctly supports the aforementioned RV32I instructions. If you are developing for the LoongArch architecture, you could not use this test, instead, you can use CLI tab to check with command lines.

Random Test Examples

1. R-Type ALU Instructions (ADD, SUB, AND, OR, XOR, SLT, SLTU, SLL, SRL, SRA)

   rd = rs1 OP rs2

   [1/64] ADD A=0xA3B1799D B=0x46685257 => 0xEA19CBF4 (expect 0xEA19CBF4) PASS

   > ALU Result = 0xA3B1799D + 0x46685257

2. U-Type Instructions (LUI)

   rd = (immediate << 12)

   [21/64] LUI(0x6C05F) A=0x00000000 B=0x00000000 => 0x6C05F000 (expect 0x6C05F000) PASS

   > ALU Result = 0x6C05F << 12 = 0x6C05F000

3. I-Type ALU Instructions (ADDI, ANDI, ORI, XORI, SLTI, SLTIU, SLLI, SRLI, SRAI)

   rd = rs1 OP immediate

   [23/64] ADDI(0x6AE) A=0x7FCD9EB1 B=0x00000000 => 0x7FCDA55F (expect 0x7FCDA55F) PASS

   > ALU Result = 0x7FCD9EB1 + 0x000006AE = 0x7FCDA55F

4. Load Instructions (LW, LH, LHU, LB, LBU)

   rd = memory[rs1 + offset]

   [41/64] LW A=0x14296C07 B=0x00000000 => 0x14296C07 (expect 0x14296C07) PASS

   > Preparation:
   >
   > Value 0x14296C07 stored at 0(gp) by uart
   >
   > Test:
   >
   > LW t2, 0(gp) // Load word into t2 register, then store in base+8
   >
   > Verification:
   >
   > Check lw Result = 0x14296C07

5. Store Instructions (SW)

   memory[rs1 + offset] = rd

   [51/64] SW A=0x5F987C71 B=0x00000000 => 0x5F987C71 (expect 0x5F987C71) PASS

   > store register value 0x5F987C71 into base+8

6. Branch Instructions (BEQ, BNE, BLT, BGE, BLTU, BGEU)

   BEQ rs1, rs2, target

   [53/64] BEQ A=0x3F76BE1D B=0x1064005C => 0x00000000 (expect 0x00000000) PASS

   > ALU result indicate weather branch is taken or not
   >
   > 0x3F76BE1D != 0x1064005C , branch not taken, result = 0

Difftest Debug implementation

TO BE RELEASED

The Difftest functionality needs to be integrated into the traditional CPU you have already built by modifying certain module interfaces and adding debug-related files. The tutorials and corresponding code for this part will be released to you later. For now, I suggest you focus entirely on developing the CPU.